

Here is the complete technical documentation for building the LSTM Fault Detection System. You can share this document directly with your team to guide the development process.

Technical Design Document: LSTM Fault Detection Model for Nuclear Reactor Safety

1. Introduction

1.1 Problem Statement

Nuclear reactors are dynamic systems where the "state" of the reactor depends heavily on what happened in the previous few seconds. Traditional fault detection systems often rely on static thresholds (e.g., "Alarm if Temp > 350°C"). However, dangerous transients like a Loss of Flow Accident (LOFA) or a Reactivity Insertion show specific *trends* (rates of change) long before they hit the absolute safety limits.

The problem is to identify these specific fault patterns in their early stages to allow for faster automated response.

1.2 Why LSTM?

We chose **Long Short-Term Memory (LSTM)** networks over traditional algorithms (like Random Forest or SVM) because:

- **Temporal Awareness:** Standard ML models view data as isolated snapshots. LSTMs view data as a "video." They understand that a temperature of 300°C is safe if it was 305°C a second ago (cooling), but dangerous if it was 250°C a second ago (rapid heating).
- **Lag Detection:** In our physics model, changes in flow affect coolant temperature, which *then* affects fuel temperature. There is a time lag. LSTMs are specifically designed to learn these time-lagged correlations.

1.3 Use Case

This model will serve as the **"Diagnostic Agent"** for the 540 MWe PHWR simulation. It will monitor live sensor streams and classify the reactor state into one of three categories: **Normal, Scram (Rod Drop), or LOFA (Pump Failure)**.

2. What is an LSTM?

2.1 Plain Language Explanation

Imagine reading a sentence. To understand the last word, you need to remember the first word. An LSTM is a neural network that has "memory." It processes data sequentially, keeping an internal "diary" of what it has seen so far. It learns what information is important to keep in the diary and what information is irrelevant and can be forgotten.

2.2 The Mechanics (Gates & Cell State)

The LSTM manages memory using three specific "Gates":

1. **Forget Gate:** Decides what information from the past is no longer useful. *Example:* If the reactor shuts down (Scram), the LSTM "forgets" the high power levels from 10 seconds ago because they are no longer relevant to the current state.
2. **Input Gate:** Decides what new information to store. *Example:* If the Temp_Fuel_ROC (Rate of Change) suddenly spikes, the gate opens to store this critical anomaly.
3. **Output Gate:** Decides what to tell the next layer. It combines the current sensor reading with the internal memory to produce a prediction.

2.3 Relation to Nuclear Physics

Our physics engine relies on differential equations (e.g., $\frac{dT_f}{dt}$). This implies that the Temperature at time t is strictly dependent on the Temperature at time $t-1$. LSTM architecture mathematically mirrors this dependency, making it the perfect tool for solving differential equation-based problems.

3. Project Overview

3.1 Objective

To build a Multi-Class Classification Model that inputs a sequence of sensor data and outputs the probability of a specific reactor state.

3.2 Output Classes

- **Class 0:** Normal Operation (Steady State).
- **Class 1:** Scram (Rapid Power Drop / Control Rod Insertion).
- **Class 2:** LOFA (Loss of Flow / Temperature Spike).

3.3 Constraints

- **Latency:** The model must make a prediction in under 100ms.
 - **Input Window:** The model requires a history of data (e.g., the last 10 seconds) to make a decision, not just the current instant.
-

4. Data Requirements

4.1 Data Source

We will use the nuclear_fault_data_enhanced.csv dataset. This file contains time-series data from multiple simulation episodes.

4.2 Input Features

The model will ingest the following continuous variables:

1. **Power:** Neutron power (Normalized).
2. **Fuel_Temp:** Average fuel temperature.
3. **Coolant_Temp:** Average coolant temperature.
4. **Pressure:** Primary heat transport system pressure.
5. **Flow:** Coolant mass flow rate.
6. **Derived Features:** Rate of Change (ROC) columns for Power and Temperature (crucial for detecting speed of accidents).

4.3 Target Variable

- **Column:** Label
- **Format:** Categorical integer (0, 1, 2). This must be One-Hot Encoded for training.

4.4 Data Shape (The 3D Tensor)

This is the most critical requirement. LSTMs do not accept 2D spreadsheets (Rows × Columns). They require a 3D Tensor:

$(\text{Samples}, \text{Time Steps}, \text{Features})$

- **Samples:** Number of "windows" or examples we extract.
 - **Time Steps:** How far back the model looks (e.g., 10 data points).
 - **Features:** The number of sensor columns (e.g., 6 features).
-

5. Model Architecture

We will construct a **Stacked LSTM** architecture.

Layer 1: Input Layer

- **Shape:** (Time_Steps, Num_Features)
- **Purpose:** Receives the sliding window of sensor data.

Layer 2: LSTM Layer (The Feature Extractor)

- **Units:** 64 Neurons.
- **Return Sequences:** True.
- **Explanation:** This layer looks at the sequence and outputs a *new* sequence of processed features. It passes the temporal structure to the next layer.

Layer 3: LSTM Layer (The Summarizer)

- **Units:** 32 Neurons.
- **Return Sequences:** False.

- **Explanation:** This layer looks at the sequence coming from Layer 2 and compresses it into a single "Summary Vector" representing the state of the reactor for that window.

Layer 4: Dropout Layer

- **Rate:** 0.2 (20%).
- **Purpose:** Randomly ignores 20% of neurons during training. This forces the model to learn robust patterns and prevents it from memorizing noise.

Layer 5: Output Dense Layer

- **Units:** 3 (corresponding to Normal, Scram, LOFA).
- **Activation:** Softmax.
- **Purpose:** Converts the summary vector into probabilities (e.g., [0.1, 0.8, 0.1] sum to 1.0).

Configuration

- **Loss Function:** Categorical Crossentropy (Standard for multi-class classification).
 - **Optimizer:** Adam (Adaptive Moment Estimation - handles noisy gradients well).
 - **Metrics:** Accuracy.
-

6. Step-by-Step Model Building

This section outlines the workflow for the development team.

Step 1: Environment Setup

Ensure the Python environment has the standard Deep Learning stack: TensorFlow (or Keras), Pandas for data manipulation, NumPy for array operations, and Scikit-Learn for preprocessing tools.

Step 2: Data Loading & Scaling

- Load the Enhanced CSV.
- **Normalization:** You **must** scale the data. Power is ~1.0, but Fuel Temp is ~1000.0. If you feed this directly, the LSTM will fail to converge. Use a MinMaxScaler to squeeze all features between 0 and 1.
- *Note:* Fit the scaler *only* on the Training data to avoid data leakage, then transform the Test data.

Step 3: Sequence Creation (Sliding Windows)

This is the hardest part of data prep. You must write a function that:

1. Takes the scaled dataset.
2. Iterates through it with a fixed window size (e.g., 10 steps).
3. **Constraint:** The window must **not** cross between different Episodes. If Episode 1 ends at row 600, you cannot have a window that contains row 599 and row 601.
4. Creates two arrays:
 - X: The 3D volume of input windows.
 - y: The target label corresponding to the *last* timestamp of each window.

Step 4: Train-Test Split

Split the episodes, not the rows.

- **Training Set:** Episodes 0 to 79.
- **Testing Set:** Episodes 80 to 99.
- This ensures the model is tested on simulation runs it has truly never seen before.

Step 5: Model Definition & Compilation

Define the architecture described in Section 5 using the Keras Sequential API. Compile the model with the Adam optimizer and Categorical Crossentropy loss.

Step 6: Model Training

- **Batch Size:** 32 or 64. (Updates weights after seeing 32 windows).
- **Epochs:** 20 to 50. (How many times it sees the full dataset).
- **Validation Split:** Use 20% of training data to monitor performance during training.
- **Early Stopping:** Implement a callback to stop training if the validation loss stops improving (prevents overfitting).

Step 7: Evaluation

Run the trained model on the Test Set. Generate a **Confusion Matrix**.

- *Success Criterion:* The model should distinguish LOFA from Normal with >95% accuracy.
- *Failure Check:* Check if the model confuses "Scram" with "Normal" during the very first second of the rod drop (this is expected, as the drop takes time to register).

7. Model Usage (Inference)

Once the model is trained and saved (e.g., lstm_nuclear_v1.h5), here is how it is used in the project:

1. **Buffer:** The system maintains a "Live Buffer" of the last N seconds of sensor readings (where N is the window size used in training).
2. **Input:** At every new time step, the buffer updates (First-In-First-Out).
3. **Process:** The content of the buffer is scaled (using the saved Scaler) and reshaped into (1, N , Features).
4. **Prediction:** The LSTM predicts the probabilities.
5. **Decision Logic:**
 - IF Probability(LOFA) > 0.8 THEN Trigger Alarm "PUMP FAILURE"
 - IF Probability(Scram) > 0.8 THEN Status "REACTOR SHUTDOWN"

8. MVP Deployment Concept

For the Minimum Viable Product (MVP) demonstration:

- The Pipeline:

Simulation (Python) $\rightarrow$ Data Buffer $\rightarrow$ LSTM Prediction $\rightarrow$ Dashboard Display

- **Integration:** The LSTM will run in a separate thread alongside the Physics Engine.
- **Visualization:** A Matplotlib or PyGame window will show the "Real" state vs. the "AI Predicted" state in real-time.
- **Limitation:** This MVP detects only the 3 fault types it was trained on. It cannot categorize a fault it hasn't seen (e.g., a pipe leak would likely be misclassified as a LOFA or Normal).

9. Conclusion

This LSTM model serves as the intelligent "eyes" of our autonomous system. By processing sequences of data rather than snapshots, it captures the physics-based trends of the reactor.

Team Next Steps:

1. **Data Team:** Generate the sequences (Step 3).
2. **Model Team:** Build the Keras architecture (Step 5).
3. **Integration Team:** Link the model output to the alert system (Section 8).